

20. Iterator Design Pattern in Java

20.1 Introduction

The **Iterator Design Pattern** is a **behavioral pattern** that provides a way to access elements of a collection object sequentially without exposing its underlying representation.

20.2 Problem it Solves

Allows traversal of complex data structures (e.g., arrays, trees, linked lists) without exposing internal structure or breaking encapsulation.

20.3 What is Iterator Pattern?

It provides a standard way to traverse through a collection of objects without exposing the underlying structure.

20.4 Real-World Analogy

A **TV remote** that lets you go channel by channel (next/prev) without opening the internals of the TV.

20.5 Structure

```
package com.mannemlokes;

public interface Iterator<T> {
    boolean hasNext();

    T next();
}

interface IterableCollection<T> {
    Iterator<T> createIterator();
}
```

20.6 Benefits

- Simplifies traversal logic
- Promotes encapsulation
- Supports multiple concurrent traversals

20.7 Implementation Steps

1. Create an Iterator interface
2. Implement concrete iterator
3. Create an IterableCollection interface
4. Implement concrete collection

20.8 Examples

Example1: Name Collection Iterator

```
package com.mannemlokes.example1;

interface Iterator<T> {
```

```

        boolean hasNext();

        T next();
    }

    interface Container<T> {
        Iterator<T> getIterator();
    }

    class NameRepository implements Container<String> {
        private String[] names = { "Alice", "Bob", "Charlie" };

        public Iterator<String> getIterator() {
            return new NameIterator();
        }

        private class NameIterator implements Iterator<String> {
            int index = 0;

            public boolean hasNext() {
                return index < names.length;
            }

            public String next() {
                return hasNext() ? names[index++] : null;
            }
        }
    }

    public class Example1IteratorClient {
        public static void main(String[] args) {
            NameRepository repo = new NameRepository();
            Iterator<String> iterator = repo.getIterator();

            while (iterator.hasNext()) {
                System.out.println(iterator.next());
            }
        }
    }

```

Output:

```

Alice
Bob
Charlie

```

Example2: Custom List with Filtered Iteration

```

package com.mannemlokes.example2;

import java.util.ArrayList;
import java.util.List;

class Book {
    String title;
    boolean isAvailable;

    public Book(String title, boolean isAvailable) {

```

```

        this.title = title;
        this.isAvailable = isAvailable;
    }
}

interface BookIterator {
    boolean hasNext();

    Book next();
}

class Library {
    List<Book> books = new ArrayList<>();

    public void addBook(Book book) {
        books.add(book);
    }

    public BookIterator availableBooksIterator() {
        return new AvailableBookIterator();
    }

    private class AvailableBookIterator implements BookIterator {
        private int index = 0;

        public boolean hasNext() {
            while (index < books.size()) {
                if (books.get(index).isAvailable)
                    return true;
                index++;
            }
            return false;
        }

        public Book next() {
            return hasNext() ? books.get(index++) : null;
        }
    }
}

public class Example2LibraryClient {
    public static void main(String[] args) {
        Library library = new Library();
        library.addBook(new Book("Design Patterns", true));
        library.addBook(new Book("Java in Depth", false));
        library.addBook(new Book("Effective Java", true));

        BookIterator iterator = library.availableBooksIterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next().title);
        }
    }
}

```

Output:

```

Design Patterns
Effective Java

```

Example3: Tree Traversal Using Iterator (In-order)

```

package com.mannemlokes.example3;

import java.util.Iterator;
import java.util.Stack;

class TreeNode {
    int val;
    TreeNode left, right;

    public TreeNode(int val) {
        this.val = val;
    }
}

class InOrderIterator implements Iterator<TreeNode> {
    private Stack<TreeNode> stack = new Stack<>();

    public InOrderIterator(TreeNode root) {
        pushLeft(root);
    }

    private void pushLeft(TreeNode node) {
        while (node != null) {
            stack.push(node);
            node = node.left;
        }
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

    public TreeNode next() {
        TreeNode node = stack.pop();
        pushLeft(node.right);
        return node;
    }
}

public class Example3TreeClient {
    public static void main(String[] args) {
        TreeNode root = new TreeNode(4);
        root.left = new TreeNode(2);
        root.right = new TreeNode(5);
        root.left.left = new TreeNode(1);
        root.left.right = new TreeNode(3);

        Iterator<TreeNode> it = new InOrderIterator(root);
        while (it.hasNext()) {
            System.out.print(it.next().val + " ");
        }
    }
}

```

Output:

```
1 2 3 4 5
```

20.9 Iterator vs Other Patterns

Pattern	Purpose
Iterator	Traverse a collection without exposing it
Composite	Treat group of objects like a single one
Visitor	Add operations without changing classes

20.10 Pros and Cons



Pros

- Clean traversal logic
- Multiple iterators can exist on same collection
- Supports lazy traversal



Cons

- May require many classes
- Slight overhead for simple collections

20.11 When to Use / Not to Use

Use When:

- You need uniform access to different collections
- You want to abstract traversal logic

Avoid When:

- Your collection is trivial (just use a loop)

20.12 Summary

Feature	Description
Intent	Traverse collections in standard way
Use Case	Collection iteration, data structures
Key Benefit	Encapsulation of traversal logic

The **Iterator Pattern** is key to separating iteration logic from the collection structure and supports powerful traversal strategies.